



II Maratona de Programação do DCOMP
Caderno de Problemas

2025

Olá!

Prova preparada com muito empenho por **Gustavo Aragão** e **Lucas Andrade**, membros do *Grupo de Estudos em Programação Competitiva - Greedy*, para a *11ª Semana Acadêmico-Cultural (SEMAC XI)* da *Universidade Federal de Sergipe (UFS - São Cristóvão)*.

Esperamos que gostem!

Good luck & Have Fun

Agradecimentos

Agradecemos especialmente aos professores **Giovanny Lucero** e **Felipe de Carvalho Pereira** pelo apoio fundamental na realização do evento e na elaboração da prova.

Também expressamos nossa gratidão à *Liga Acadêmica de Empreendedorismo – Innovation* pelo suporte contínuo na organização e na divulgação do evento e ao *Grupo de Extensão em Maratonas de Programação da Universidade Federal de Alagoas - GEMA-UFAL* por terem testado as questões da maratona.

Agradecemos ainda a **Carlos Daniel** e **Cauan Santos Silva** pelo auxílio na infraestrutura do *judge*, bem como aos “*melancias*” — **Natália Andrade**, **Vênisson dos Santos**, **Gabriel Marinho**, **João da Silva** e **Jairo Dantas** — por toda a colaboração durante a realização do evento.

Informações Gerais

Esta prova contém 16 páginas (excluindo a capa), numeradas de 1 a 16. Verifique se a prova está completa.

A) Sobre os problemas

- 1) Para soluções em *C/C++* e *Python*, o nome do arquivo-fonte não é significativo, pode ser qualquer nome.
- 2) Se sua solução é em *Java*, ela deve ser chamada *codigo_de_problema.java* onde *codigo_de_problema* é a letra maiúscula que identifica o problema. Lembre que em *Java* o nome da classe principal deve ser igual ao nome do arquivo.

B) Sobre a entrada

- 1) A entrada deve ser lida da entrada padrão.
- 2) Todas as linhas da entrada, incluindo a última, terminam com o caractere de fim-de-linha (`\n`).
- 3) Quando a entrada contém vários valores separados por espaço, existe exatamente um espaço entre dois valores consecutivos na mesma linha.

C) Sobre a saída

- 1) A saída deve ser escrita na saída padrão.
- 2) A saída deve ser impressa no formato especificado pelo problema. A saída não deve conter nenhum dado adicional.
- 3) Todas as linhas da saída, incluindo a última, terminam com o caractere de fim-de-linha (`\n`).

Problema A. Amigo

Vamos supor que temos um amigo que gosta bastante de matemática. Ultimamente, este nosso amigo vem passando horas mergulhado no estudo de funções recursivas e, como era de se esperar, acabou se deparando com a famosa sequência de Fibonacci, que é definida da seguinte forma:

$$fib(0) = 1$$

$$fib(1) = 1$$

$$fib(n) = fib(n - 1) + fib(n - 2), \text{ para } n \geq 2$$

Inspirado com seus novos conhecimentos, ele tentou criar um programa para calcular o valor de $fib(K)$ para um K qualquer. No entanto, ele rapidamente percebeu que seu código recursivo simples estava demorando uma eternidade para valores um pouco maiores de K .

Sabendo que você é craque na programação, ele veio até você pedindo ajuda. Ele precisa de um programa eficiente que consiga calcular $fib(K)$.

Como esses números crescem muito rápido e podem se tornar gigantescos, seu amigo pediu para que o resultado fosse o valor final módulo $10^9 + 7$.

Dica: Em algumas linguagens de programação, o operador de módulo é representado pelo símbolo `%`. O módulo nada mais é do que o resto da divisão inteira de um número por outro. Por exemplo: o cálculo $7 \% 5$ resulta em 2, pois se dividirmos 7 por 5, o resto da divisão é 2.

Entrada

A única linha da entrada contém um inteiro K , onde $0 \leq K \leq 10^5$.

Saída

Exiba o valor de $fib(K)$ módulo $10^9 + 7$.

Todas as linhas da saída, incluindo a última, terminam com o caractere de fim-de-linha (`\n`).

<i>Exemplo de Entrada 1</i> 5	<i>Exemplo de Saída 1</i> 5
<i>Exemplo de Entrada 2</i> 10	<i>Exemplo de Saída 2</i> 55

Problema B. Bons Caminhos

Rogério, um grande adepto da filosofia dos "Bons Caminhos", sempre se preocupa em escolher as melhores rotas em suas viagens. Desta vez, ele tem um desafio e decidiu pedir ajuda a você, um(a) especialista em Teoria dos Grafos.

Com isso, ele te entregou um mapa com N cidades e M estradas que as conectam. Sua tarefa é descobrir se o professor Rogério pode ir de sua cidade atual (cidade 1) até seu destino (cidade N). Se for possível, ele quer encontrar a rota que passe pelo menor número de cidades (incluindo a origem e o destino), para otimizar sua jornada.

Entrada

A primeira linha de entrada tem dois inteiros N e M ($2 \leq N \leq 10^5$, $1 \leq M \leq 2 \times 10^5$), representando o número de cidades e o número de estradas, respectivamente. As cidades são numeradas $1, 2, \dots, N$. A cidade atual do professor Rogério é a 1 e seu destino é a N .

A seguir, há M linhas descrevendo as estradas. Cada linha tem dois inteiros A e B ($1 \leq A, B \leq N$), que sinalizam que existe uma estrada de mão dupla entre as cidades A e B . Cada estrada conecta duas cidades distintas, e há no máximo uma estrada entre quaisquer duas cidades.

Saída

Se for possível viajar da cidade 1 para a N , imprima primeiro K : o número mínimo de cidades na rota. Se não houver rotas, imprima "MAUS CAMINHOS!" (sem aspas).

Todas as linhas da saída, incluindo a última, terminam com o caractere de fim-de-linha ($\backslash n$).

<i>Exemplo de Entrada 1</i>	<i>Exemplo de Saída 1</i>
5 5 1 2 1 3 2 3 3 4 4 5	4

<i>Exemplo de Entrada 2</i>	<i>Exemplo de Saída 2</i>
3 1 1 2	MAUS CAMINHOS!

Problema C. Café e Corrida

Gilton é conhecido no circuito de competições tanto por seus algoritmos eficientes quanto por sua resistência nas corridas de rua. Ele é, de fato, um programador e corredor experiente.

Para sua próxima competição, a Maratona Internacional do Código (e da Cafeína), Gilton precisa planejar sua estratégia de hidratação. A maratona possui N estações de hidratação ao longo do percurso.

Como todo programador de elite, Gilton sabe que o segredo do desempenho não está só nos treinos, mas na cafeína. Ele decidiu que, durante a corrida, só irá parar nas estações que oferecem seu combustível essencial: o **café**. Se a estação oferecer qualquer outra coisa (água, isotônico, chá), ele passará direto, otimizando seu tempo e seu *buffer* de cafeína.

Sua tarefa é simples: dado o mapa das estações de hidratação, determine quantos copos de café Gilton terá bebido ao final da corrida, assumindo que ele bebe exatamente um copo em cada estação de café em que para.

Entrada

A primeira linha da entrada contém um único inteiro N ($1 \leq N \leq 100$), o número total de estações de hidratação na maratona.

A segunda linha contém N inteiros E_i ($1 \leq i \leq N$), separados por espaços, representando o tipo de bebida em cada estação i .

Os tipos de bebida são codificados da seguinte forma: **1** simboliza *água*, **2** simboliza *isotônico* e **3** simboliza *café*.

Saída

Imprima um único inteiro, o número total de copos de café que Gilton bebeu durante a maratona.

Todas as linhas da saída, incluindo a última, terminam com o caractere de fim-de-linha ($\backslash n$).

Exemplo de Entrada 1 5 1 3 2 1 3	Exemplo de Saída 1 2
Exemplo de Entrada 2 7 1 2 1 1 2 1 2	Exemplo de Saída 2 0

Problema D. Diversão na SEMAC

A tão esperada XI Semana Acadêmico-Cultural da UFS (SEMAC) está se aproximando! Gabrielzinho, um calouro recém-chegado, já está com os olhos brilhando para o cronograma. O evento está lotado com N atividades incríveis: palestras, workshops, apresentações das ligas acadêmicas e, o melhor de tudo, a **Maratona de Programação!**

Infelizmente para o nosso amigo Gabrielzinho, é impossível comparecer a todas as atividades devido ao choque de horários. Mesmo assim, nada irá impedi-lo de aproveitar a semana ao máximo! Para ele, “aproveitar ao máximo” significa participar do maior número possível de eventos, do começo ao fim.

O problema é que a quantidade de atividades é grande e, para piorar, Gabrielzinho ainda está “patinando” um pouco na disciplina de Algoritmos. Incapaz de otimizar o seu roteiro sozinho, ele decidiu recorrer à única pessoa que poderia salvá-lo: ninguém mais, ninguém menos que você, um(a) veterano(a) de computação, uma lenda das maratonas de programação e um(a) exímio(a) conhecedor(a) da SEMAC.

Ele entregou a lista completa das N atividades e pediu sua ajuda para montar o “roteiro perfeito”. Sua tarefa é simples: diga a ele o número máximo de eventos que ele consegue assistir.

Entrada

A primeira linha da entrada contém um inteiro N ($1 \leq N \leq 10^5$), o número total de atividades na SEMAC.

Cada uma das N linhas seguintes descreve uma atividade. Cada linha contém dois inteiros, S_i e E_i ($1 \leq S_i \leq E_i \leq 10^5$), o tempo de início e fim da atividade, respectivamente.

Você só pode escolher uma atividade se ela não colidir com outra atividade já escolhida. Uma atividade A e uma atividade B colidem se seus intervalos de tempo tiverem qualquer sobreposição. Se uma atividade termina no tempo T e outra começa no tempo T , considere que elas não colidem (Gabrielzinho consegue sair correndo de uma para a outra).

Saída

Imprima um inteiro K , o número máximo de atividades que Gabrielzinho consegue assistir.

Todas as linhas da saída, incluindo a última, terminam com o caractere de fim-de-linha ($\ln$).

<i>Exemplo de Entrada 1</i>	<i>Exemplo de Saída 1</i>
10 6 7 4 5 8 9 2 3 10 11 1 2 9 10 3 4 5 6 7 8	10

Problema E. Entre Linhas de Fogo

Kalil e André estão treinando para um campeonato de *Counter-Strike*. Durante as partidas, eles passaram a discutir estratégias avançadas de marcação do mapa. Uma das ideias de Kalil foi marcar, no mini-mapa, linhas horizontais e verticais representando corredores, ângulos de visão e trajetórias de granadas.

André, sempre competitivo, sugeriu transformar isso em um desafio: “*Vamos ver quem consegue contar mais rápido quantos cruzamentos existem entre essas linhas no mapa!*”.

Como ambos são muito teimosos e não chegaram a um acordo, pediram sua ajuda para resolver o problema de forma automática.

As linhas do mini-mapa que eles estão desenhando são sempre horizontais ou verticais. Além disso, as linhas paralelas entre si nunca são coincidentes e nenhum ponto final é um ponto de interseção com outra reta.

Dado o conjunto de linhas que Kalil e André desenharam, determine quantos pontos de interseção existem entre as linhas horizontais e verticais.

Entrada

A primeira linha contém um inteiro N ($1 \leq N \leq 10^5$) representando a quantidade de segmentos.

Cada uma das próximas N linhas contém quatro inteiros: x_1, y_1, x_2 e y_2 , representando um segmento que começa no ponto (x_1, y_1) e termina no ponto (x_2, y_2) .

É garantido que todos os segmentos são horizontais ou verticais e as extremidades são pontos distintos no espaço, ou seja, $(x_1, y_1) \neq (x_2, y_2)$.

É garantido que $(-10^6 \leq x_1 \leq x_2 \leq 10^6)$ e $(-10^6 \leq y_1 \leq y_2 \leq 10^6)$.

Saída

Imprima um único número inteiro que representa o **número total de pontos de interseção** entre os segmentos horizontais e verticais.

Todas as linhas da saída, incluindo a última, terminam com o caractere de fim-de-linha ($\backslash n$).

<i>Exemplo de Entrada 1</i>	<i>Exemplo de Saída 1</i>
3 2 3 7 3 3 1 3 5 6 2 6 6	2

<i>Exemplo de Entrada 2</i>	<i>Exemplo de Saída 1</i>
5 0 2 6 2 3 0 3 5 1 4 5 4 4 1 4 6 2 3 2 7	5

Problema F. Fila do Resun



“Restaurante Universitário (Resun) da Universidade Federal de Sergipe (UFS), Campus São Cristóvão”. Foto por Portal Infonet.
Disponível em: <https://infonet.com.br/noticias/educacao/restaurante-universitario-da-ufs-volta-a-funcionar-na-quinta-30/>

O Restaurante Universitário (Resun) é famoso no campus por suas longas filas na hora do almoço. O tempo de espera costuma ser tão grande que muitos estudantes chegam atrasados para as aulas do período da tarde.

Para tentar diminuir o tamanho da fila e reduzir o tempo de espera, a administração do Resun implementou uma nova estratégia: em vez de um único local de serviço, eles criaram múltiplos guichês. Um "guichê" é simplesmente um ponto de atendimento (um balcão) onde um funcionário efetua a leitura da carteirinha ou do *QR Code* do estudante.

Hoje, você vê que há N estudantes na fila e o Resun está operando com A guichês de atendimento, todos funcionando em paralelo. Você observou que cada atendente leva exatamente T minutos para servir um estudante. Assim que um atendente finaliza um atendimento, ele imediatamente chama o próximo estudante da fila.

Como um bom estudante de computação, você quer calcular qual será o tempo total de espera até que a última pessoa da fila seja atendida, para saber se a nova estratégia realmente está funcionando.

Sua tarefa é calcular quantos minutos levará para que todos os N estudantes sejam completamente atendidos.

Entrada

A entrada consiste em uma única linha contendo três números inteiros separados por espaço: N , A e T .

- $1 \leq N \leq 1000$ (número de estudantes na fila)
- $1 \leq A \leq 10$ (número de guichês de atendimento)
- $1 \leq T \leq 5$ (tempo em minutos por atendimento)

Saída

Imprima um único número inteiro que representa o tempo total, em minutos, até que todos os N estudantes sejam completamente atendidos.

Todas as linhas da saída, incluindo a última, terminam com o caractere de fim-de-linha ($\backslash n$).

<i>Exemplo de Entrada 1</i>	<i>Exemplo de Saída 1</i>
10 2 3	15

<i>Exemplo de Entrada 2</i>	<i>Exemplo de Saída 2</i>
7 3 2	6

Problema G. Gatos Furiosos



“Gatos (alunos jubilados) comendo ração pela UFS”. Foto por @gatosufs.
Disponível em: <https://www.instagram.com/p/B6HJJf0jWHG/>

O Campus de São Cristóvão da Universidade Federal de Sergipe (UFS) é mundialmente famoso por seus inúmeros (e famintos) gatos. Para mantê-los felizes e bem alimentados, o projeto de extensão "Gato Amigo" instalou N pontos de alimentação pelo campus. Como um(a) amante de gatos, você não poderia ficar de fora e se voluntariou, sendo agora o(a) responsável pela distribuição das rações.

Após alguns dias no projeto, você notou que os gatos da UFS são MUITO observadores. Se um gato percebe que um ponto de alimentação tem mais ração do que outro, é o sinal para o caos: todos os gatos correm para o ponto mais cheio, causando uma briga furiosa e generalizada. Salve-se quem puder!

Para evitar o desastre e manter a paz felina, você adotou uma regra de ouro: todos os N pontos de alimentação devem ter exatamente a mesma quantidade de porções de ração.

Infelizmente, o orçamento do projeto acabou! Você não pode comprar mais ração (adicionar ao total) nem pode jogar ração fora (remover do total). A única operação que você pode fazer é mover porções de ração de um ponto i para um ponto j . Você pode repetir essa operação quantas vezes quiser, movendo qualquer quantidade de ração entre os pontos.

Dado o número de porções em cada ponto de alimentação, sua tarefa é determinar se é possível (ou impossível) redistribuir a ração para que todos os pontos fiquem com a mesma quantidade, evitando assim a briga dos gatos.

Entrada

A primeira linha da entrada contém um inteiro N ($0 \leq N \leq 10^5$), o número de pontos de alimentação.

A segunda linha contém N inteiros separados por espaço: $r_1, r_2, \dots, r_n$ ($0 \leq r_i \leq 10^9$), onde r_i é a quantidade de porções de ração no i -ésimo ponto.

Saída

Imprima **“SIM”** (sem aspas) se for possível igualar a quantidade de ração em todos os pontos através da redistribuição. Caso contrário, imprima **“NAO”** (sem aspas e sem acento).

Todas as linhas da saída, incluindo a última, terminam com o caractere de fim-de-linha ($\backslash n$).

<i>Exemplo de Entrada 1</i>	<i>Exemplo de Saída 1</i>
5 7 4 12 5 2	SIM

<i>Exemplo de Entrada 2</i>	<i>Exemplo de Saída 2</i>
2 1 2	NAO

Problema H. Hardwares Harmônicos

Os professores Calebe e Botto estão no Laboratório de Hardware da UFS fazendo uma limpeza. Eles encontraram uma caixa cheia de placas de circuito antigas e precisam separar quais ainda servem para as aulas práticas.

Para testar, eles usam uma placa modelo que está configurada perfeitamente. Cada placa possui uma fila de N pequenos interruptores. Podemos representar o estado de cada interruptor com um dígito binário: 1 para ligado e 0 para desligado. A sequência desses bits forma um número inteiro em notação binária.

Por exemplo, se uma placa tem a configuração *101* (Ligado, Desligado, Ligado), isso corresponde ao cálculo: $1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 5$. Assim, dizemos que a configuração dessa placa é simplesmente o número 5 em notação decimal.

Para avaliar as peças, eles seguem o princípio da **harmonia**: quanto mais a placa se parece com a modelo, melhor. Para isso, eles estabeleceram um nível de **tolerância** T . O teste consiste em contar em quantas posições os interruptores (bits) diferem entre a placa testada e a modelo:

- Se a quantidade de diferenças estiver dentro da **tolerância** (ou seja, for menor ou igual a T), a placa é considerada harmônica e ainda serve.
- Se o número de diferenças passar de T , ela está fora do padrão e é descartada.

Como a caixa está cheia de placas, os professores precisam de uma ajudinha: **eles querem que você desenvolva um algoritmo capaz de contar quantas delas realmente são harmônicas.**

Dica: Em C/C++, Python e Java, os **operadores bitwise** para o AND, OR, XOR e NOT são, respectivamente, definidos por: $\&$, $|$, $\wedge$ e $\sim$.

Entrada

A primeira linha contém três inteiros N , Q e T ($0 \leq N \leq 20$, $1 \leq Q \leq 1000$, $0 \leq T \leq N$), representando, respectivamente:

- N : O número de interruptores (bits) em cada placa;
- Q : A quantidade de placas antigas na caixa para testar;
- T : A Tolerância máxima de diferenças permitida.

A segunda linha contém um único inteiro M ($1 \leq M \leq 2^N$), representando a configuração da placa Modelo.

As próximas Q linhas contém, cada uma, um inteiro P_i ($0 \leq P_i < 2^N$), representando a configuração de uma placa antiga da caixa.

Saída

Imprima um único inteiro: a quantidade de placas que são consideradas harmônicas (ou seja, que têm no máximo T diferenças em relação à placa modelo M).

Todas as linhas da saída, incluindo a última, terminam com o caractere de fim-de-linha ($\backslash n$).

<i>Exemplo de Entrada 1</i>	<i>Exemplo de Saída 1</i>
5 3 1 7 15 0 6	2

Problema I. Instabilidade no SIGAA

A Universidade Federal de Sergipe (UFS) finalmente decidiu enfrentar um problema recorrente: a instabilidade eterna do SIGAA. Depois de incontáveis relatos de alunos jurando que “o sistema travou bem na hora de enviar o trabalho”, a equipe de infraestrutura concluiu que boa parte das quedas acontecia porque a rede interna do campus havia se tornado complexa demais.

Para tentar salvar o SIGAA (e a sanidade dos estudantes), o professor Ricardo, responsável pela rede, iniciou uma grande reestruturação. O *backbone* foi simplificado e agora conecta os principais setores do campus (como DCOMP, Didáticas, Resun, Reitoria...) em uma topologia de árvore. Cada ponto dessa árvore é um roteador/switch principal.

Enquanto isso, a professora Edilayne apresenta sua última invenção: um protocolo experimental de otimização de tráfego. Quando ativado em um roteador u , esse protocolo gera uma **economia de recursos** no valor de V_u . Cada roteador tem seu próprio valor de economia.

Mas há um detalhe dramático: o protocolo é tão agressivo que não pode ser ativado em dois roteadores diretamente conectados. Se isso acontecer, a rede entra em parafuso e... bom, aí o SIGAA cairia antes mesmo de abrir a tela de login.

Ricardo e Edilayne querem ativar o protocolo no maior número possível de roteadores úteis, mas sem que nenhum par de roteadores adjacentes esteja ativado ao mesmo tempo. Eles precisam saber qual é o valor máximo total de economia que é possível obter. Sua missão é ajudar nessa decisão.

Entrada

A primeira linha contém um inteiro N ($1 \leq N \leq 10^5$), o número de roteadores na rede.

Já a segunda linha contém N valores $V_1, V_2, \dots, V_N$, onde V_i ($1 \leq V_i \leq 10^9$) é a economia gerada ao ativar o protocolo no roteador i .

As próximas $N-1$ linhas contém dois inteiros u e v , indicando que existe uma conexão direta entre os roteadores u e v .

A rede sempre forma uma árvore.

Saída

Imprima um único inteiro: o **valor máximo total de economia** possível, escolhendo um conjunto de roteadores onde **nenhum par de roteadores adjacentes esteja selecionado**.

Todas as linhas da saída, incluindo a última, terminam com o caractere de fim-de-linha ($\ln$).

Exemplo de Entrada 1 4 100 40 40 40 1 2 1 3 1 4	Exemplo de Saída 1 120
Exemplo de Entrada 2 5 10 100 50 100 10 1 2 2 3 3 4 4 5	Exemplo de Saída 2 200

Problema J. Jogando com o Adversário

Breno é um estudante brilhante de Ciência da Computação e é absolutamente imbatível no “Jogo do Lema do Bombeamento” (aquela disputa teórica sobre linguagens regulares que tira o sono de muitos estudantes).

Cansado de vencer sempre usando a mesma estratégia de dividir palavras em uvw , Breno decidiu que precisava de novos ares. Ele procurou seu eterno rival (vamos chamá-lo apenas de *Adversário*) e propôs um desafio diferente. As regras são simples, mas a estratégia é profunda:

- Existem N pedras espalhadas em um tabuleiro 100×100 .
- A posição de cada pedra é dada por coordenadas (l, c) , onde l é a linha e c a coluna.
- Breno e o adversário jogam alternadamente. Breno sempre começa (afinal, é ele quem está propondo o jogo).
- Em cada jogada, o jogador deve escolher **uma** pedra e movê-la para uma nova posição, **aproximando-a da origem $(0,0)$** .
- Dado uma pedra em (l, c) , o jogador escolhe um inteiro $u > 0$ e move a pedra para uma das seguintes posições (desde que não saia do tabuleiro):
 - $(l - u, c)$ — movimento vertical para baixo;
 - $(l, c - u)$ — movimento horizontal para a esquerda;
 - $(l - u, c - u)$ — movimento diagonal para a esquerda e para baixo.
- Várias pedras podem ocupar a mesma posição simultaneamente.
- O jogo termina quando não for mais possível realizar movimentos (todas as pedras chegaram em $(0,0)$). O jogador que fizer o **último movimento** vence o jogo.

Breno, confiante em sua capacidade de cálculo, acredita que sempre vencerá se o jogo permitir. O adversário, porém, não é bobo e também jogará de maneira ótima (perfeita). Sua tarefa é determinar quem será o vencedor, dada a configuração inicial das pedras.

Entrada

A primeira linha da entrada contém um inteiro N ($1 \leq N \leq 1000$) representando o número de pedras no tabuleiro.

As N linhas seguintes descrevem a posição de cada pedra. A i -ésima linha contém dois inteiros l_i e c_i , onde ($1 \leq l_i, c_i \leq 100$), indicando a linha e a coluna onde a i -ésima pedra está.

Saída

Imprima **“Breno”** (sem aspas) se Breno vencer o jogo, ou **“Adversario”** (sem aspas e sem acento) se o adversário vencer, assumindo que ambos jogam de forma ótima.

Todas as linhas da saída, incluindo a última, terminam com o caractere de fim-de-linha ($\ln$).

Exemplo de Entrada 1 2 1 3 2 3	Exemplo de Saída 1 Breno
Exemplo de Entrada 1 1 1 2	Exemplo de Saída 1 Adversario

Problema K. Kung-Fu Sergipano



“Mestre Bruno Mao”

O Professor Bruno Mao não é apenas um mestre em algoritmos, mas também o grão-mestre do lendário **Kung-Fu Sergipano**, uma arte marcial antiga desenvolvida nas terras do DCOMP. Durante seus treinos no tatame (que curiosamente fica dentro do laboratório), ele costuma ensinar sequências complexas de golpes para seus alunos.

Sempre que termina de demonstrar uma sequência perfeita, ele olha para a turma com um sorriso confiante e solta sua frase de efeito: “*Isso é muito fácil!*”. O problema é que, para os alunos, não é nada trivial.

Hoje, Bruno demonstrou a **Sequência Mestre**, uma série de movimentos representados por letras maiúsculas ('C' para Chute, 'S' para Soco, 'D' para Defesa). Em seguida, um único aluno foi escolhido para ir à frente e tentar reproduzir os golpes.

Para evitar humilhar o aluno na frente de todos, o professor decidiu ser generoso. Ele entende que, no calor da batalha, o aluno pode perder o foco momentaneamente.

Por isso, a regra de aprovação é flexível. O professor vai procurar se existe uma **substring*** na sequência de golpes do aluno que satisfaça as seguintes condições:

1. A **substring*** válida deve começar com um **prefixo não vazio** da Sequência Mestre.
2. Logo após esse prefixo, pode haver **no máximo UM movimento errado** (um intervalo de “descuido” de tamanho 0 ou 1).
3. Imediatamente após esse erro (ou imediatamente após o prefixo, se não houve erro), deve vir o restante da Sequência Mestre (**sufixo não vazio**), completando-a.

Importante: O aluno pode ter feito vários movimentos errados **antes** de começar a acertar e **depois** de terminar. O que importa é encontrar essa estrutura “Prefixo Mestre + (0 ou 1 Erro) + Sufixo Mestre” como uma **substring*** válida da sequência do aluno.

** Uma substring de uma string **S** é uma sequência contígua de caracteres de **S**. Por exemplo, “gip”, “Ser” e “ergi” são alguns exemplos de substring da palavra “Sergipe”.*

Entrada

Na primeira linha, você recebe a *string* **M** da **Sequência Mestre**, que pode ter até 10^5 caracteres. A string contém apenas os caracteres 'C', 'S' e 'D'.

A segunda linha contém a *string* **S**, a sequência de golpes do aluno, que pode ter até 10^5 caracteres. A string contém apenas os caracteres 'C', 'S' e 'D'.

Saída

Para cada aluno, você deve imprimir **“MUITO FACIL”** (sem aspas e sem acento) se for possível encontrar a **Sequência Mestre** com **até no máximo UM erro** como uma substring da sequência do aluno. Caso contrário, imprima **“NAO MUITO FACIL”** (sem aspas e sem acento).

Todas as linhas da saída, incluindo a última, terminam com o caractere de fim-de-linha (\n).

<i>Exemplo de Entrada 1</i>	<i>Exemplo de Saída 1</i>
CCSS DDCCDSSDD	MUITO FACIL

Nota do Exemplo 1:

A Sequência Mestre é CCSS. A sequência do aluno é DDCCDSSDD.

Na sequência do aluno, podemos identificar a substring CCDSS (índices 2 a 6), onde:

- CC: Prefixo da Mestre.
- D: 1 movimento errado (tamanho 1, permitido).
- SS: Sufixo da Mestre.

O padrão foi encontrado.

<i>Exemplo de Entrada 2</i>	<i>Exemplo de Saída 2</i>
SCD DDSSCCCCDSSS	NAO MUITO FACIL

Nota do Exemplo 2:

A Sequência Mestre é SCD.

O aluno executou ...SCCCCCD...

Ele fez o prefixo “S”, depois errou 4 vezes (CCCC), e completou com “CD”. Como o intervalo de erro tem tamanho 4 (maior que o permitido, que é 1), essa tentativa é inválida. Não há outra substring que satisfaça a condição.

Problema L. Lanchinhos Favoritos

O jovem professor Felipe está preparando sua lancheira para mais um dia de aula na Universidade Federal de Sergipe (UFS).

Em seu armário, tem à sua disposição N lanchinhos diferentes. Cada lanche possui um peso e um valor de *favoritismo*, que indica o quanto Felipe gosta dele. Entretanto, infelizmente sua lancheira possui um peso máximo suportado de X , o que torna impossível levar todos os lanchinhos de uma vez.

Observando a situação, ele percebeu logo que estava diante de um dos problemas mais clássicos da área de Otimização, o famoso Problema da Mochila.

Felipe decidiu então que seria uma boa oportunidade levar essa situação para a sala de aula. Com isso, ele pediu a sua ajuda para escrever um programa que o ajude a descobrir qual o **maior** valor total de *favoritismo* que ele pode obter, de modo que o peso total dos lanches escolhidos não ultrapasse o limite da lancheira, lembrando que cada lanche só pode ser escolhido uma única vez.

Entrada

A primeira linha da entrada contém dois inteiros N e X ($1 \leq N \leq 1000$, $1 \leq X \leq 10^5$), representando o número de lanchinhos disponíveis e o peso máximo que a lancheira suporta, respectivamente.

A segunda linha contém N inteiros $p_1, p_2, \dots, p_N$, onde $1 \leq p_i \leq 1000$ representa o peso de cada lanche.

A terceira linha contém N inteiros $f_1, f_2, \dots, f_N$, onde $1 \leq f_i \leq 1000$ representa o valor de favoritismo de cada lanche.

Saída

Imprima um único inteiro que representa o maior valor total de *favoritismo* que Felipe pode obter sem ultrapassar o limite de peso da lancheira.

Todas as linhas da saída, incluindo a última, terminam com o caractere de fim-de-linha ($\backslash n$).

<i>Exemplo de Entrada 1</i>	<i>Exemplo de Saída 1</i>
4 10 4 8 5 3 5 12 8 1	13

<i>Exemplo de Entrada 2</i>	<i>Exemplo de Saída 2</i>
5 12 3 6 4 2 5 6 10 5 3 8	19

Nota de pesar:

É com profundo pesar que anunciamos o desaparecimento da lancheira do Professor Felipe 😞.

Problema M. Mercado Municipal



“Mercado Municipal Antônio Franco”

Disponível em: <https://turismosergipe.com.br/conheca-a-diversidade-de-cores-sabores-e-de-cultura-dos-mercados-municipais-de-aracaju/>

O Mercado Municipal Antônio Franco é um dos ícones arquitetônicos e culturais de Aracaju. Situado às margens do Rio Sergipe, é o lugar perfeito para encontrar o melhor do artesanato sergipano, desde redes e bordados até peças de cerâmica e lembrancinhas temáticas.

Hoje, no entanto, o clima no mercado é de pura correria! Um enorme navio de cruzeiro acaba de atracar no porto e um guia turístico ligou avisando que um grupo gigantesco de turistas está a caminho. Eles querem comprar uma quantidade exata de T lembrancinhas idênticas (pequenas réplicas do Relógio do Mercado) para levar de recordação.

A associação dos artesãos do mercado reuniu N artesãos habilidosos para dar conta dessa encomenda. Cada artesão tem seu próprio ritmo de trabalho: você sabe exatamente quantos segundos cada um leva para produzir uma única lembrancinha.

Sabendo que todos os artesãos podem trabalhar simultaneamente (em paralelo) e que eles começam a trabalhar no mesmo instante, sua tarefa é ajudar a associação a descobrir qual é o tempo mínimo necessário para que as T lembrancinhas estejam prontas.

Entrada

A primeira linha da entrada contém dois inteiros N e T ($1 \leq N \leq 2 \times 10^5$, $1 \leq T \leq 10^9$), o número de artesãos disponíveis e o total de lembrancinhas encomendadas.

A segunda linha contém N inteiros $k_1, k_2, \dots, k_N$, onde $1 \leq k_i \leq 10^9$ representa o tempo (em segundos) que o i -ésimo artesão leva para fazer uma única lembrancinha.

Saída

Imprima um único número inteiro que representa o tempo mínimo (em segundos) necessário para produzir as T lembrancinhas.

Todas as linhas da saída, incluindo a última, terminam com o caractere de fim-de-linha ($\ln$).

Exemplo de Entrada 1 3 7 3 2 5	Exemplo de Saída 1 8
Exemplo de Entrada 2 10 10 6 5 1 2 1 5 10 4 6 6	Exemplo de Saída 2 4